

PIF Interim Presentation 2nd semester

By: Zaslavskyi Dmytro



Table of Contents:

- 1) Sensors 2 – brightness and barometric sensors...3
- 2) Web Database Server...8
- 3) Assembly...25
- 4) Data Transfer...31
- 5) Lighting...36
- 6) Backup...43

PIF – Sensors 2

By: Zaslavskyi Dmytro

Table of contents:

- 1) Item List
- 2) Soldering
- 3) Results
- 4) Conclusion

List of Items:

The list of items for this sprint includes:

1x Barometric Sensor,

1x Light Sensor,

1x 4 pin adapter with wiring,

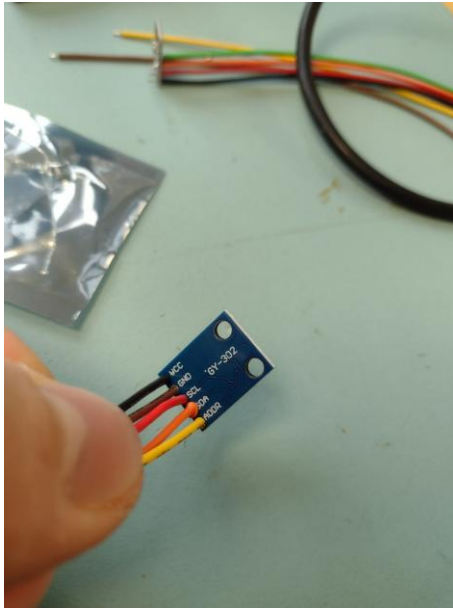
1x 6 pin adapter with wiring,

1x Raspberry Pi with PCB board attached to it and a screen,

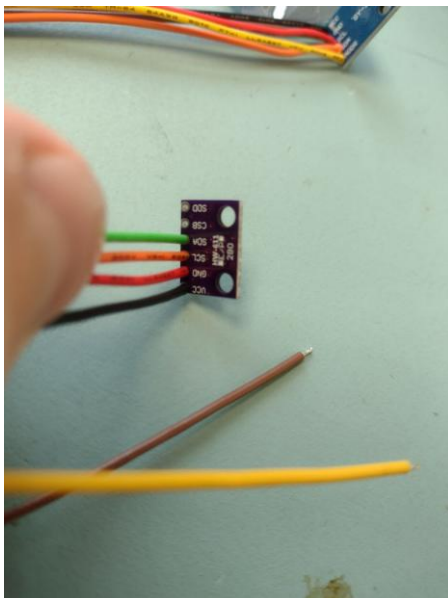
1x Soldering station.

Soldering:

To solder together the light sensor, I have used 4 pin connector with wiring. Following the document's guide on the socket connection, I checked the wiring colors and connected them to the corresponding sockets

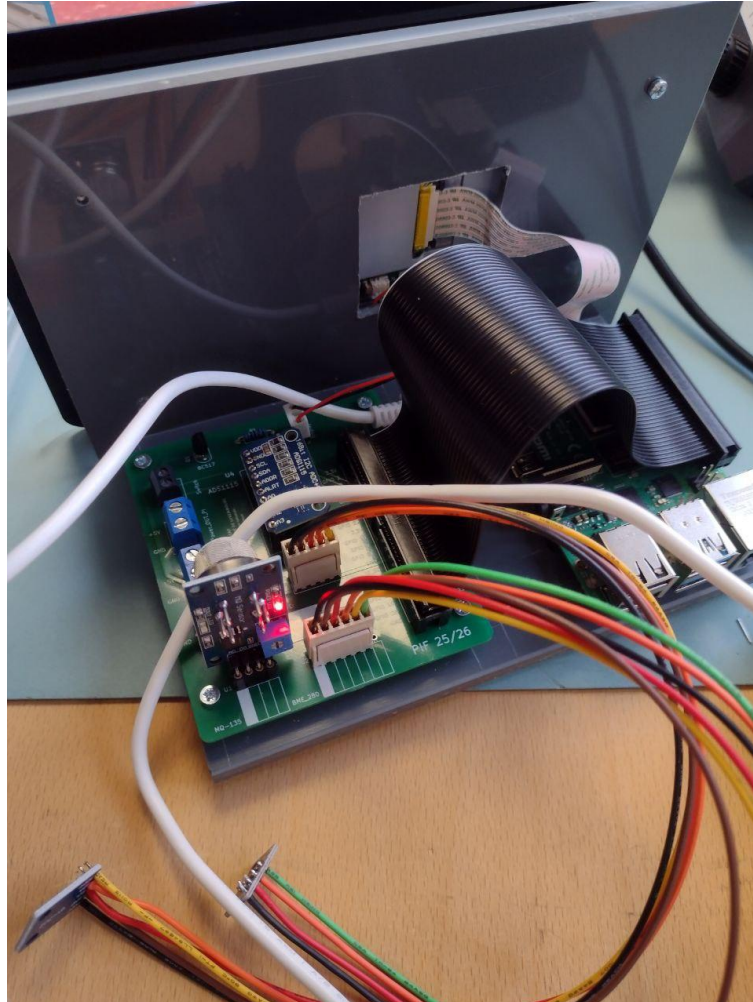


After that I did the same with the gas sensor, but the document has shown that 2 other wires should remain disconnected from the gas sensor.



Results:

This is the following result that I have received.
Unfortunately, due to the lack of time we could not properly check whether the sensors worked.



Conclusion:

The main real problem in the task was to correctly identify the arrangement of cabling, in order to allow me to solder correct sockets on the sensors. Because we did not have time to work on our raspberry pi and check its functionality I will have to do it later, without inclusion to the report.

Userstory A4 – Web Database Server

By: Zaslavskyi Dmytro



Table of Contents:

1. Introduction. pg 3
2. Installation of Apache 2, PHP, assigning permissions to folders, configuration and deployment of the virtualhost. pg 4-6
3. Installation of mysql-server, mysql_secure_installation, creation of management users for phpMyAdmin. pg 7-8
4. Installation of phpMyAdmin, configuration and connection to mysql-server. pg 9-12
5. Deployment of the database and website. pg 13
6. Setup of a self-signed certificate(SSL). pg 14-16
7. Conclusion. pg 17

1.Introduction:

This documentation includes the necessary configuration steps to set up a web-database server on the Ubuntu Desktop Operative System that distributes the website to the users and allows administrators to

2. Installation of Apache 2, PHP, assigning permissions to folders, configuration and deployment of the virtualhost:

To install apache2 and php, I used `sudo apt install apache2 libapache2-mod-php`. This allows to immediately install apache2.service with php module installed and enabled enabled. Check the installation with `sudo systemctl status apache2`. This allows us to check if the service is running.

```
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-03-08 12:57:07 CET; 3h 13min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 1408 (apache2)
    Tasks: 11 (limit: 4542)
   Memory: 74.8M (peak: 75.7M)
      CPU: 4.682s
   CGroup: /system.slice/apache2.service
           └─1408 /usr/sbin/apache2 -k start
             └─1414 /usr/sbin/apache2 -k start
               └─1415 /usr/sbin/apache2 -k start
                 └─1416 /usr/sbin/apache2 -k start
                   └─1417 /usr/sbin/apache2 -k start
                     └─1418 /usr/sbin/apache2 -k start
                       └─3539 /usr/sbin/apache2 -k start
                         └─3585 /usr/sbin/apache2 -k start
                           └─3586 /usr/sbin/apache2 -k start
                             └─3587 /usr/sbin/apache2 -k start
                               └─3655 /usr/sbin/apache2 -k start

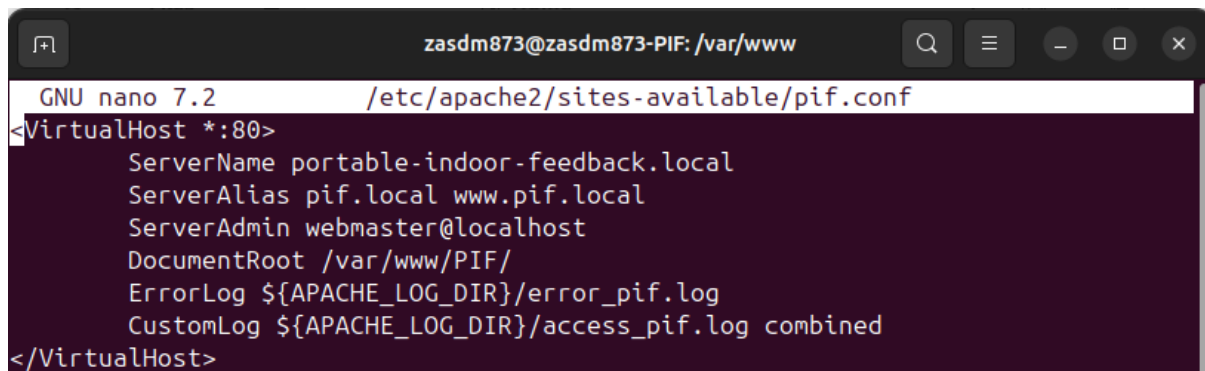
Mar 08 12:57:06 zasdm873-PIF systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 08 12:57:07 zasdm873-PIF apachectl[1401]: AH00557: apache2: apr_sockaddr_info_get() failed for zasdm873-PIF
Mar 08 12:57:07 zasdm873-PIF apachectl[1401]: AH00558: apache2: Could not reliably determine the server's fully qualified
Mar 08 12:57:07 zasdm873-PIF systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Installation is successful. That means that we can proceed to the next step. Next we need to create a directory for our website. This is why we run `sudo mkdir /var/www/PIF`. After creating directory called PIF at /var/www – we use `cd /var/www` and there we use `sudo chown www-data:www-data PIF && sudo chmod 755 PIF`. Running this command set (marked by “&&” that means AND) will change the ownership of PIF folder to www-data of group www-data (user and group created by Apache2) and change the permissions to read-write-execute for the owner, read-execute for group members, read-execute for other users.

Then we run `ls -la` to see the owner and permissions of the directory PIF.

```
zasdm873@zasdm873-PIF:/var/www$ ls -la
total 16
drwxr-xr-x  4 root    root    4096 Mar  4 09:22 .
drwxr-xr-x 15 root    root    4096 Mar  4 09:07 ..
drwxr-xr-x  2 root    root    4096 Mar  4 09:07 html
drwxr-xr-x  3 www-data www-data 4096 Mar  4 10:41 PIF
```

Success. Now, we must create and enable new website by creating a virtualhost. To do this we use `sudo nano /etc/apache2/sites-available/pif.conf`. This will start writing(with “nano” console text editor) file called pif.conf in the directory of /etc/apache2/sites-available where all configurations are set, in the /apache2/sites-enabled are only running configurations. In pif.conf we write the following configurations:



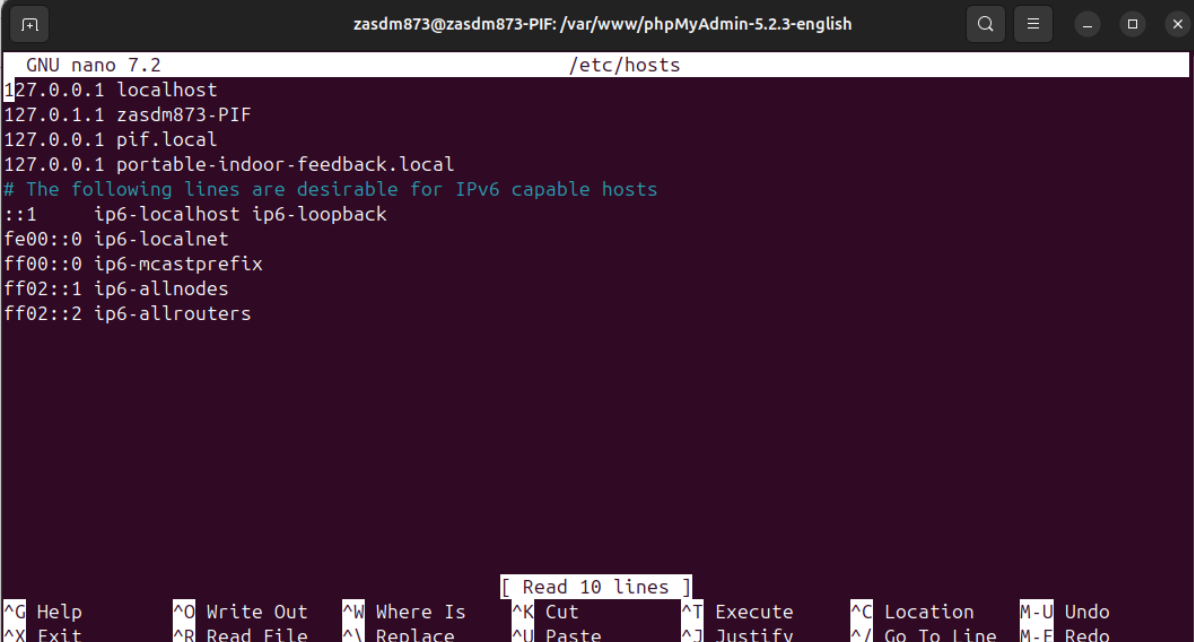
```
GNU nano 7.2 /etc/apache2/sites-available/pif.conf
<VirtualHost *:80>
    ServerName portable-indoor-feedback.local
    ServerAlias pif.local www.pif.local
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/PIF/
    ErrorLog ${APACHE_LOG_DIR}/error_pif.log
    CustomLog ${APACHE_LOG_DIR}/access_pif.log combined
</VirtualHost>
```

`<VirtualHost *:80>` tells to create a virtual host listening to any port and respond with port 80. Servername assigns the website a permanent name that is recognized by apache2. ServerAlias creates additional names that are recognized by apache2 and will redirect it to the ServerName. ServerAdmin defines the email address the server includes in error messages and server-generated documents sent to clients. DocumentRoot defines the directory from which Virtualhost will upload the website. ErrorLog and

CustomLog will be used to assign log files for our webserver in order to streamline the maintenance. After creating the config we save it and before enabling it we use `sudo apache2 configtest` that will run all our configuration files and test for any errors.

```
zasdm873@zasdm873-PIF: /var/www$ sudo apache2ctl configtest
[sudo] password for zasdm873:
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1.
Set the 'ServerName' directive globally to suppress this message
Syntax OK
```

Syntax test has returned okay. So next we use `sudo a2ensite pif.conf` to deploy our website and then we use `sudo systemctl reload apache2` to update the service. In this case we use reload because it will allow to not have any downtime in the server work, though restart can be used for the same purpose. To access our webpages later I have edited file `/etc/hosts` with `sudo nano /etc/hosts/` where I have added my server alias to the list of the hosts, which will allow me to access my website later.



```
zasdm873@zasdm873-PIF: /var/www/phpMyAdmin-5.2.3-english
GNU nano 7.2 /etc/hosts
127.0.0.1 localhost
127.0.1.1 zasdm873-PIF
127.0.0.1 pif.local
127.0.0.1 portable-indoor-feedback.local
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

[ Read 10 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

3. Installation of mysql-server, mysql_secure_installation, creation of administrator user for phpMyAdmin:

To install mysql-server I used command `sudo apt install mysql-server`. After the installation of the mysql I ran `sudo systemctl status mysql` to check if the installation is correct.

```
zasdm873@zasdm873-PIF:/$ sudo systemctl status mysql
Warning: The unit file, source configuration file or drop-ins of mysql.service changed on disk. Run 'systemctl daemon-reload' to reload units.
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-03-08 12:57:09 CET; 4h 46min ago
     Main PID: 1403 (mysqld)
    Status: "Server is operational"
       Tasks: 39 (limit: 4542)
      Memory: 442.1M (peak: 480.7M)
         CPU: 2min 18.808s
        CGroup: /system.slice/mysql.service
                └─1403 /usr/sbin/mysqld

Mar 08 12:57:06 zasdm873-PIF systemd[1]: Starting mysql.service - MySQL Community Server...
Mar 08 12:57:09 zasdm873-PIF systemd[1]: Started mysql.service - MySQL Community Server.
```

Then, I ran `sudo mysql` and created an admin user for future phpMyAdmin installation using sql code `CREATE USER 'admin'@'localhost' IDENTIFIED BY 'PIF2026admin'; GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost' WITH GRANT OPTION;` - this command will create additional administrator user that will be able to access and manage phpMyAdmin after installation and configuration. Then I proceeded to run `sudo mysql_secure_installation`. After running the command it deployed several prompts:

```
zasdm873@zasdm873-PIF:~$ sudo mysql_secure_installation

Securing the MySQL server deployment.

Connecting to MySQL using a blank password.

VALIDATE PASSWORD COMPONENT can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?

Press y|Y for Yes, any other key for No: y

There are three levels of password validation policy:

LOW      Length >= 8
MEDIUM  Length >= 8, numeric, mixed case, and special characters
STRONG Length >= 8, numeric, mixed case, special characters and dictionary file

Please enter 0 = LOW, 1 = MEDIUM and 2 = STRONG: 0

Skipping password set for root as authentication with auth_socket is used by default.
If you would like to use password authentication instead, this can be done with the "ALTER_USER" command.
See https://dev.mysql.com/doc/refman/8.0/en/alter-user.html#alter-user-password-management for more information.

By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.

Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.

Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : y
Success.

By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.

Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y
- Dropping test database...
Success.
- Removing privileges on test database...
Success.

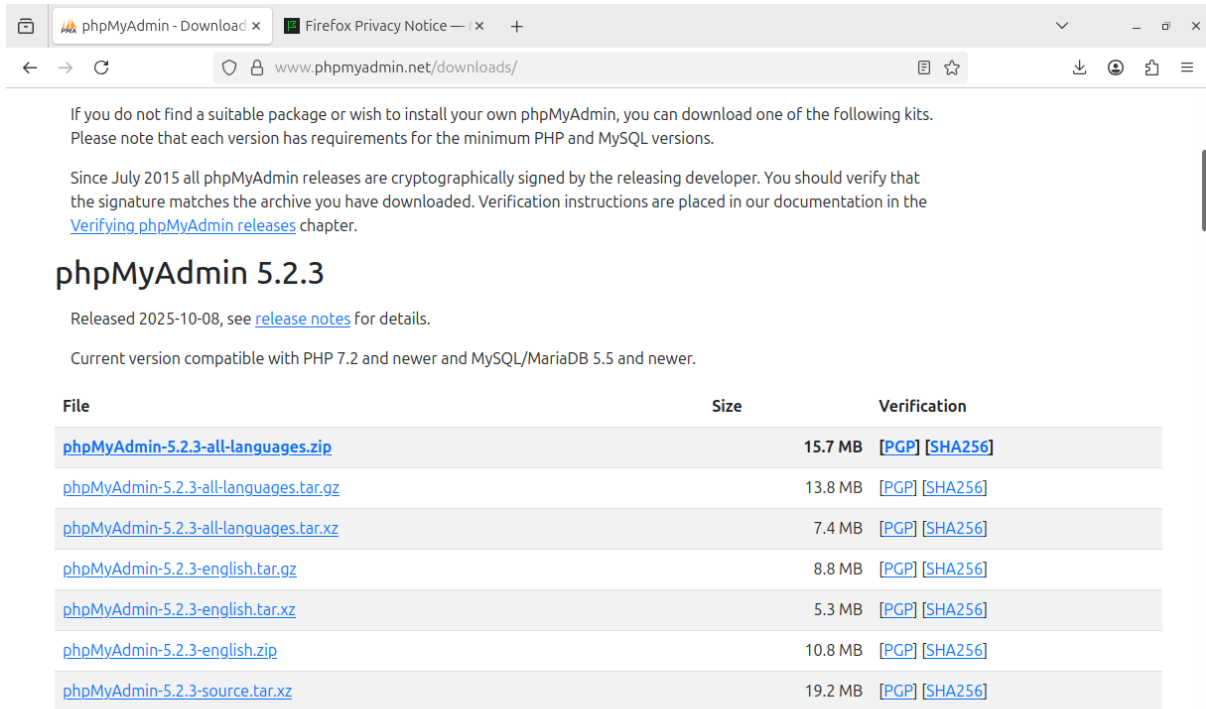
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
```

Those prompts have provided password validation, removed anonymous users(that could lead to a vulnerability in the mysql-server), disallowed remote login with root, removed test database(that could lead to a vulnerability in the system) and reloaded privileges to confirm the changes.

4. Installation of phpMyAdmin, configuration and connection to mysql-server:

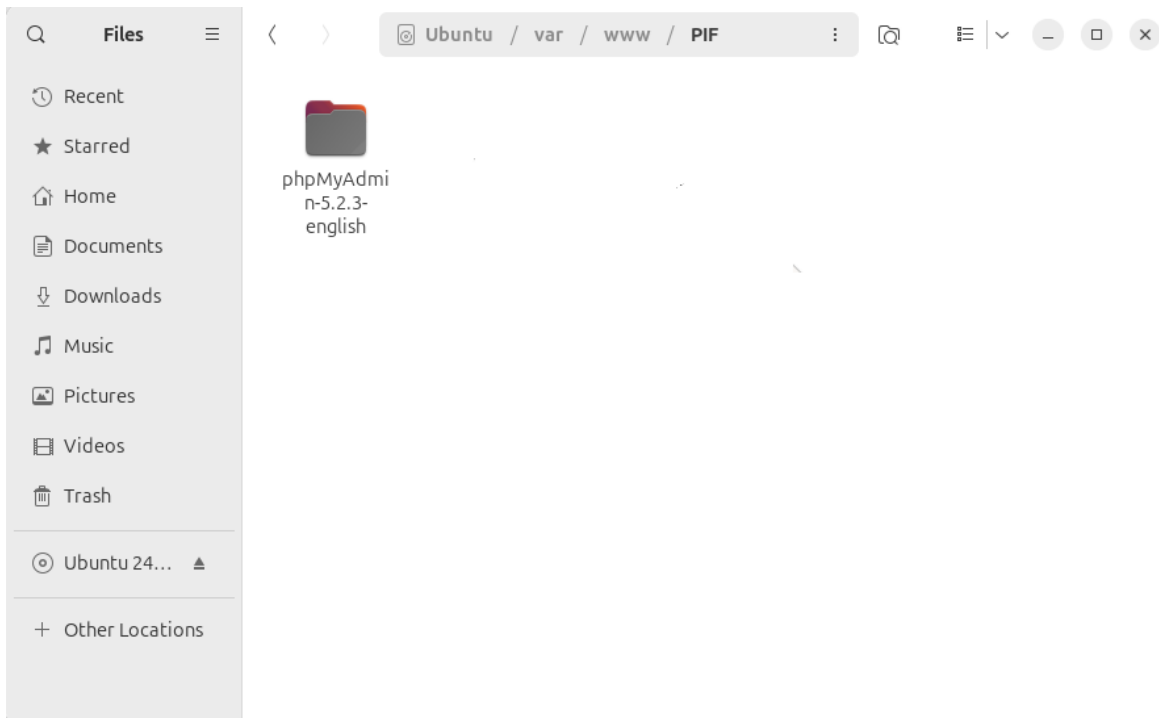
To install phpMyAdmin I have went for the official website where I downloaded a .tar file of phpMyAdmin 5.2.3

A screenshot of a web browser showing the phpMyAdmin download page. The browser's address bar shows 'www.phpmyadmin.net/downloads/'. The page content includes instructions on downloading kits, a note about cryptographic signing since July 2015, and a section for 'phpMyAdmin 5.2.3' released on 2025-10-08. Below this is a table with columns 'File', 'Size', and 'Verification'. The table lists seven download options: 'phpMyAdmin-5.2.3-all-languages.zip' (15.7 MB), 'phpMyAdmin-5.2.3-all-languages.tar.gz' (13.8 MB), 'phpMyAdmin-5.2.3-all-languages.tar.xz' (7.4 MB), 'phpMyAdmin-5.2.3-english.tar.gz' (8.8 MB), 'phpMyAdmin-5.2.3-english.tar.xz' (5.3 MB), 'phpMyAdmin-5.2.3-english.zip' (10.8 MB), and 'phpMyAdmin-5.2.3-source.tar.xz' (19.2 MB). Each entry includes links for PGP and SHA256 verification.

File	Size	Verification
phpMyAdmin-5.2.3-all-languages.zip	15.7 MB	[PGP] [SHA256]
phpMyAdmin-5.2.3-all-languages.tar.gz	13.8 MB	[PGP] [SHA256]
phpMyAdmin-5.2.3-all-languages.tar.xz	7.4 MB	[PGP] [SHA256]
phpMyAdmin-5.2.3-english.tar.gz	8.8 MB	[PGP] [SHA256]
phpMyAdmin-5.2.3-english.tar.xz	5.3 MB	[PGP] [SHA256]
phpMyAdmin-5.2.3-english.zip	10.8 MB	[PGP] [SHA256]
phpMyAdmin-5.2.3-source.tar.xz	19.2 MB	[PGP] [SHA256]

phpMyAdmin 4.9.11

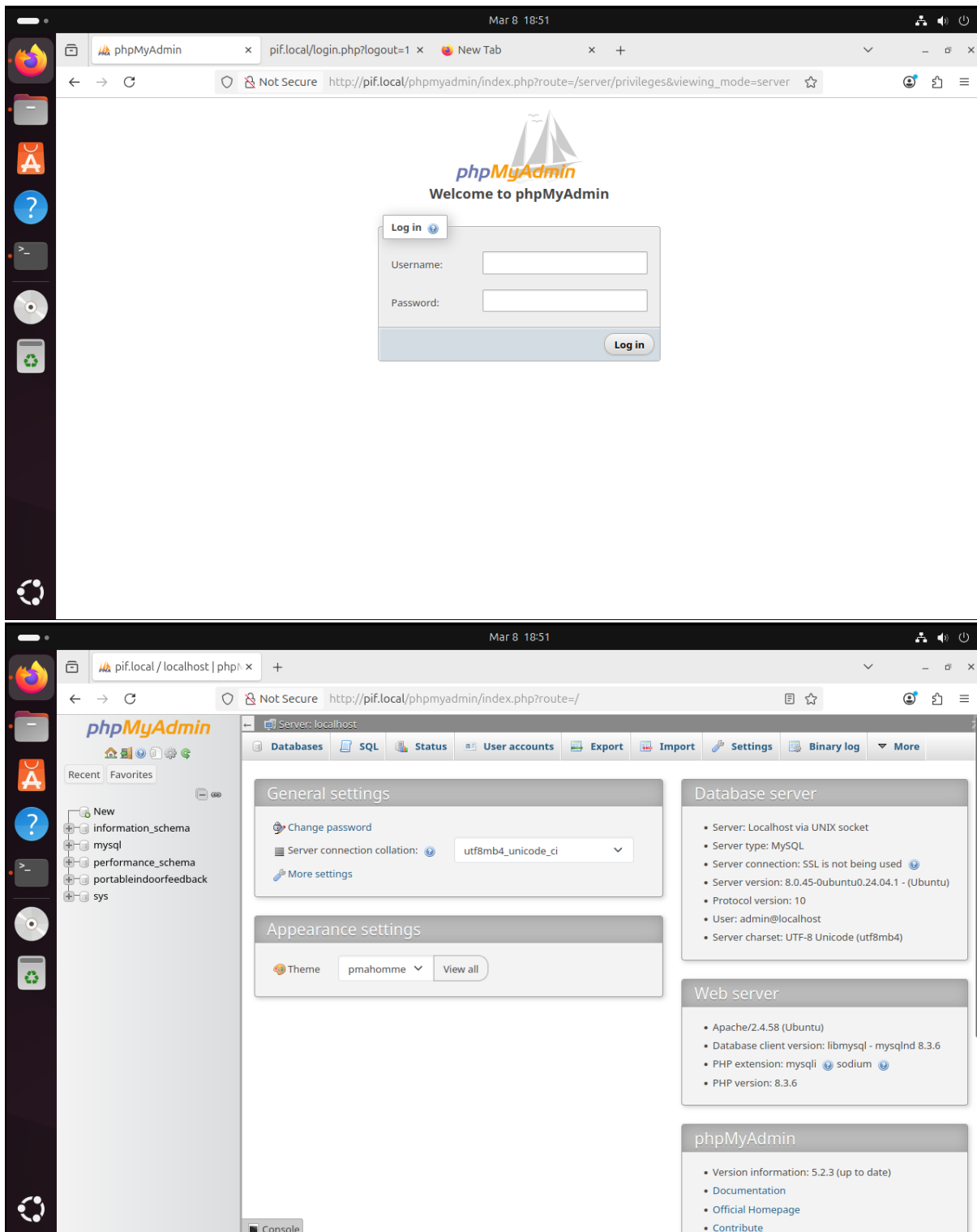
Then, I unpacked the file and moved it into the PIF directory with `sudo cp -r ~/Downloads/phpMyAdmin-5.2.3-english /var/www/PIF`



Then I have renamed it with `sudo mv phpMyAdmin-5.2.3-english phpmyadmin` to have easier access to it. Inside the phpmyadmin directory I have renamed file `config.sample.inc.php` to `config.inc.php` with `sudo mv config.sample.inc.php config.inc.php` to provide it with base configuration file.

```
zasdm873@zasdm873-PIF:/var/www/PIF/phpmyadmin$ ls
babel.config.json  CONTRIBUTING.md  js              RELEASE-DATE-5.2.3  templates  yarn.lock
ChangeLog          doc              libraries       robots.txt         themes
composer.json      examples         LICENSE         setup              tmp
composer.lock      favicon.ico      package.json    show_config_errors.php  url.php
config.inc.php     index.php        README          sql                vendor
```

Then, I have installed mysql extension for php with `sudo apt install php8.3-mysql` to allow my webpages to run and support phpmyadmin. Now test if phpmyadmin works



After confirmation of phpmyadmin working. We can now create a new database user for our webserver.

Add user account

Login Information

User name:

Host name:

Password: Strength:

Strong

Re-type:

Authentication plugin:

Generate password:

Database for user account

☐ Create database with same name and grant all privileges.
☐ Grant all privileges on wildcard name (username,%).
☐ Grant all privileges on database portableindoorfeedback.

Global privileges ☐ Check all

Note: MySQL privilege names are expressed in English.

Data ☒ **Structure** ☐ **Administration** ☐ **Resource limits** ☐ **SSL** ☒

☒ SELECT
☒ INSERT
☒ UPDATE
☒ DELETE
☒ FILE

☐ CREATE
☐ ALTER
☐ INDEX
☐ DROP
☐ CREATE TEMPORARY TABLES
☐ SHOW VIEW
☐ CREATE ROUTINE
☐ ALTER ROUTINE
☐ EXECUTE
☐ CREATE VIEW
☐ EVENT
☐ TRIGGER

☐ GRANT
☐ SUPER
☐ PROCESS
☐ RELOAD
☐ SHUTDOWN
☐ SHOW DATABASES
☐ LOCK TABLES
☐ REFERENCES
☐ REPLICATION CLIENT
☐ REPLICATION SLAVE
☐ CREATE USER

Note: Setting these options to 0 (zero) removes the limit.

MAX QUERIES PER HOUR

MAX UPDATES PER HOUR

MAX CONNECTIONS PER HOUR

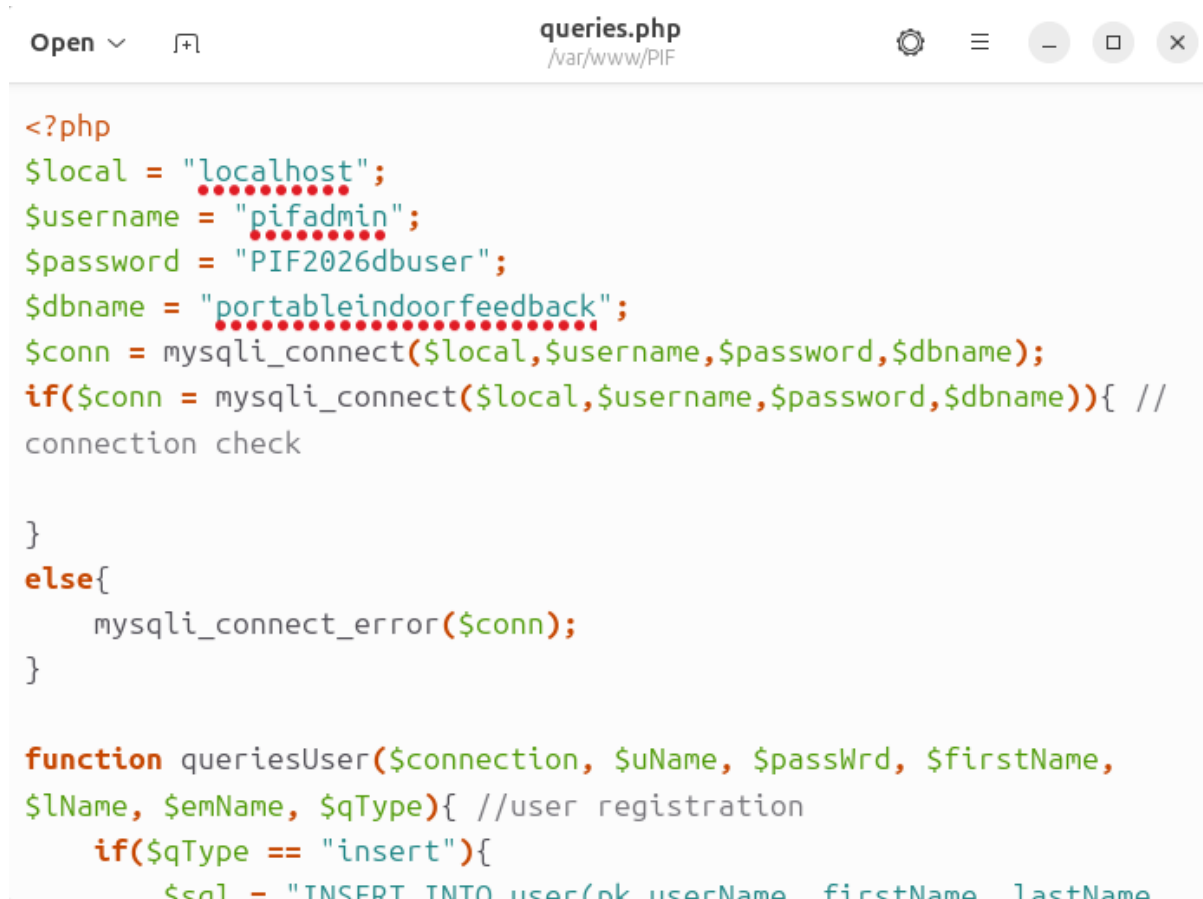
MAX USER_CONNECTIONS

☒ REQUIRE NONE
☐ REQUIRE SSL

Current website only requires a managing user that can manipulate data.

5. Deployment of the website and database:

To upload my website and database, as a test I have used my past solutions for previous sprints. I adjusted the configuration to use my new database user and uploaded database sql.



```
<?php
$local = "localhost";
$username = "pifadmin";
$password = "PIF2026dbuser";
$dbname = "portableindoorfeedback";
$conn = mysqli_connect($local,$username,$password,$dbname);
if($conn = mysqli_connect($local,$username,$password,$dbname)){ //
connection check

}
else{
    mysqli_connect_error($conn);
}

function queriesUser($connection, $uName, $passWrd, $firstName,
$lName, $emName, $qType){ //user registration
    if($qType == "insert"){
        $sql = "INSERT INTO user (pk userName firstName lastName
```

6. Creation of a self-signed certificate:

In this section I will create a self-signed certificate for the web-database server. To prepare for the key deployment, we need to open /etc/ssl and create new folder to contain the certificates called PIF_keys.

Firstly we need to generate an rsa key. We use `sudo openssl genrsa -out pif.local.key 2048`. Then, we need to create a certificate with `sudo openssl req -x509 -new -nodes -key pif.local.key -sha256 -days 365 -out pif.local.crt`. In this command req means certificate request, x509 output a certificate instead of a csr file that needs to be signed. After this a series of prompts will pop-up:

```
zasdm873@zasdm873-PIF:/etc/ssl/PIF_keys$ sudo openssl req -x509 -new -nodes -new -nodes -key pif.local.key -sha256 -days 365 -out pif.local.crt
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:LU
State or Province Name (full name) [Some-State]:NA
Locality Name (eg, city) []:Differdange
Organization Name (eg, company) [Internet Widgits Pty Ltd]:LPEM
Organizational Unit Name (eg, section) []:NA
Common Name (e.g. server FQDN or YOUR name) []:portable-indoor-feedback.local
Email Address []:zasdm873@school.lu
```

Those prompts request information for certificate. After that, we have to enable ssl in apache2 with `a2enmod ssl`. That is used to enable modules of apache2. After enabling ssl, we need to configure a new virtualhost and deploy it. The configuration for new virtualhost is as follows:

```
GNU nano 7.2 /etc/apache2/sites-available/pif-ssl.conf
<VirtualHost *:443>
    ServerName portable-indoor-feedback.local
    ServerAlias pif.local www.pif.local
    ServerAdmin webmaster@localhost
    SSLEngine on
    SSLCertificateFile /etc/ssl/PIF_keys/pif.local.crt
    SSLCertificateKeyFile /etc/ssl/PIF_keys/pif.local.key
    DocumentRoot /var/www/PIF/
    ErrorLog ${APACHE_LOG_DIR}/error_pif.log
    CustomLog ${APACHE_LOG_DIR}/access_pif.log combined
</VirtualHost>

[ File '/etc/apache2/sites-available/pif-ssl.conf' is unwritable ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

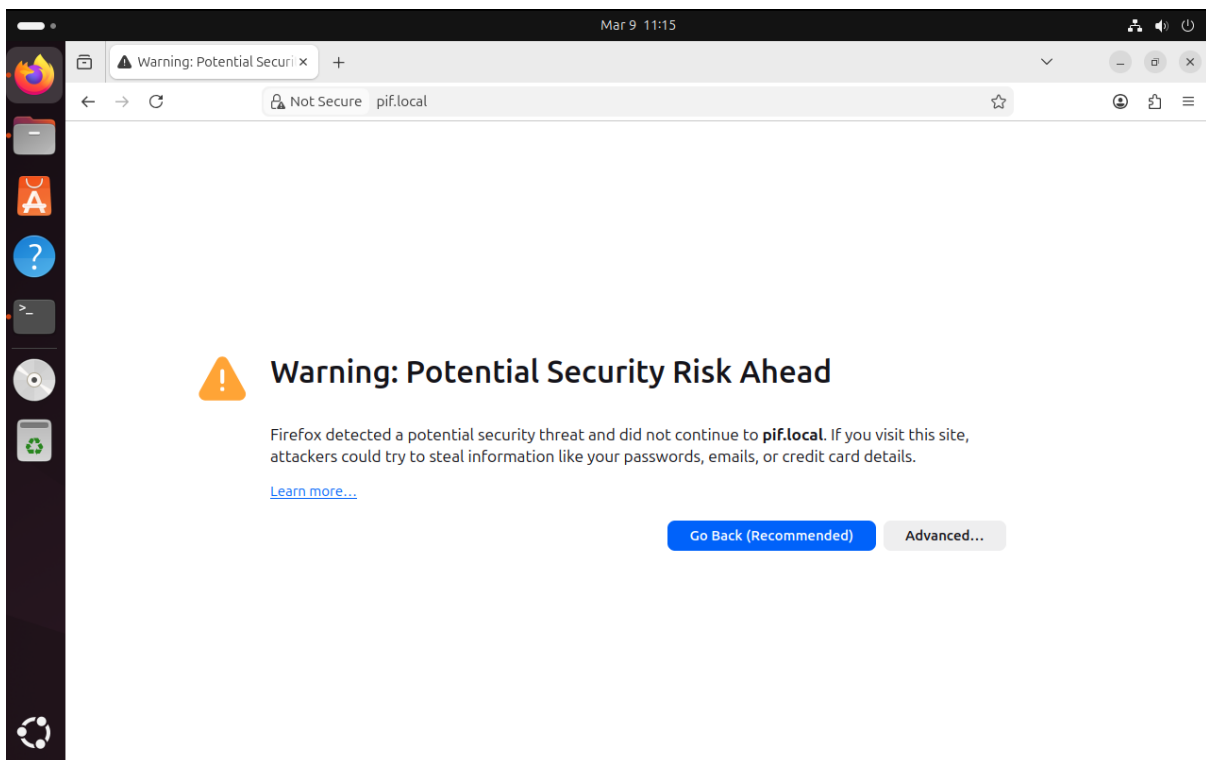
The only thing added is the SSLEngine – SSLCertificateFile and SSLCertificateKeyFile that enable usage of ssl and select the ssl certificate that is used to represent the website. After that we setup the redirect in old pif.conf:

```
GNU nano 7.2 /etc/apache2/sites-available/pif.conf
<VirtualHost *:80>
    ServerName portable-indoor-feedback.local
    ServerAlias pif.local www.pif.local
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/PIF/
    Redirect permanent / https://portable-indoor-feedback.local
    ErrorLog ${APACHE_LOG_DIR}/error_pif.log
    CustomLog ${APACHE_LOG_DIR}/access_pif.log combined
</VirtualHost>

[ File '/etc/apache2/sites-available/pif.conf' is unwritable ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

We add new line *Redirect permanent* with reference to https version of the website in order to avoid leaving vulnerable access to the webserver. After that we test the

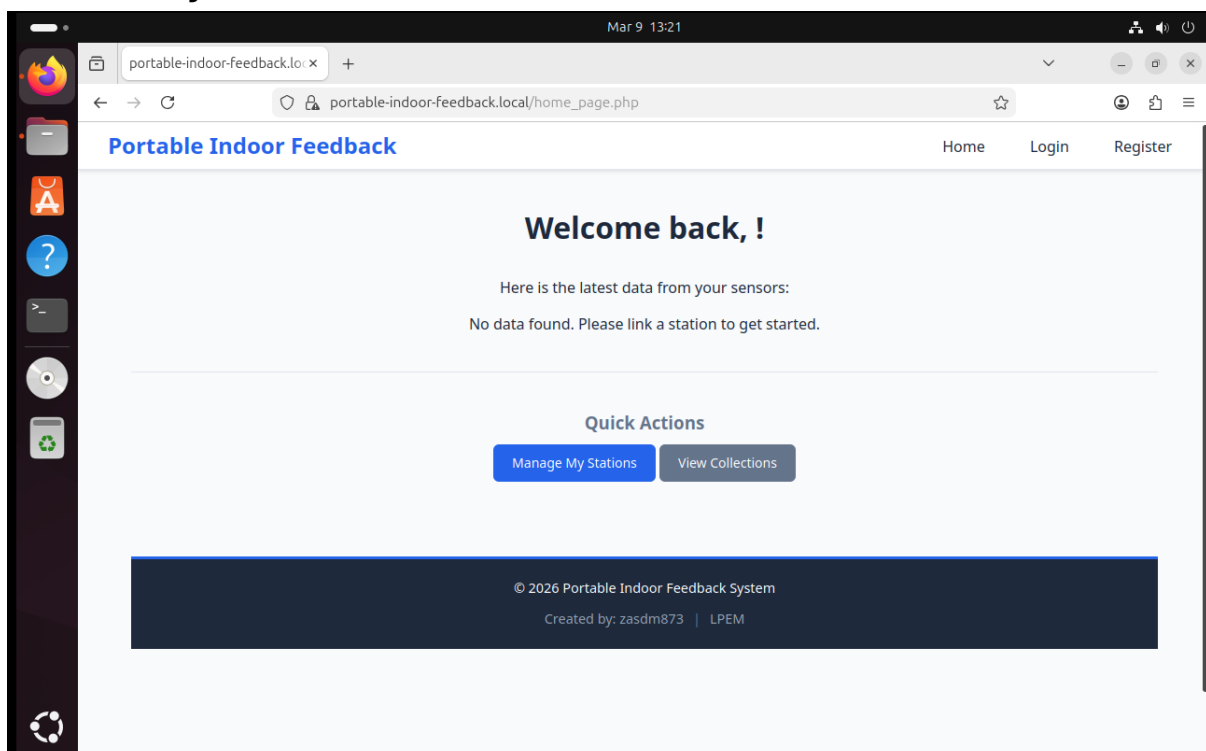
certificate:



We got a warning, that means that our self-signed certificate work because browser recognizes our certificate as self signed.

7. Conclusion:

In the end of this documentation we get a working web-database server that can provide website access to other users. It is reinforced with database users that have restricted access, secure installation of mysql-server and supports https port that encrypts traffic – making sniffing less likely.



Userstory A6 – Assembly

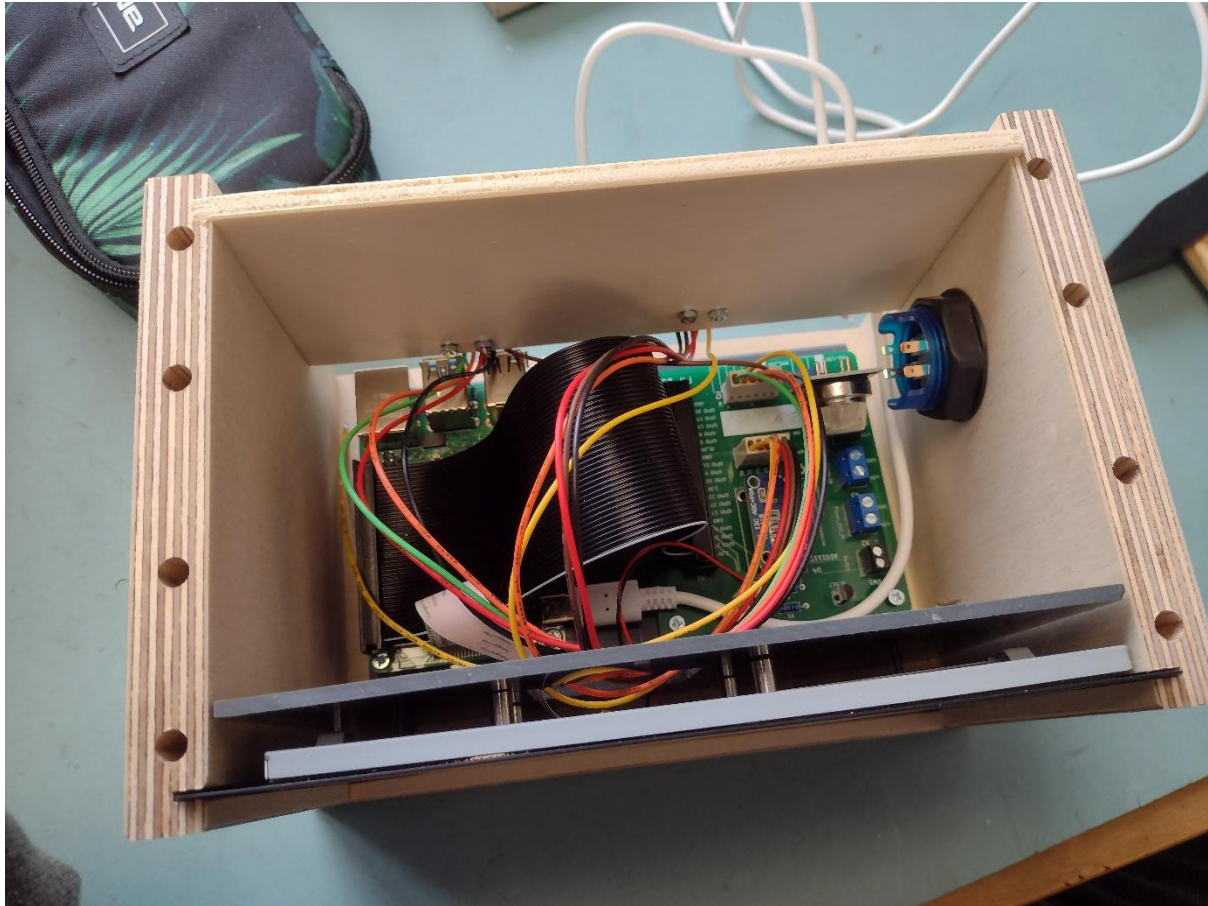


Table of contents:

1. Introduction
2. Holes
3. Assembling
4. Result

Introduction:

In this document will be described steps that were taken to assemble the container for the Raspberrypi. Item list included

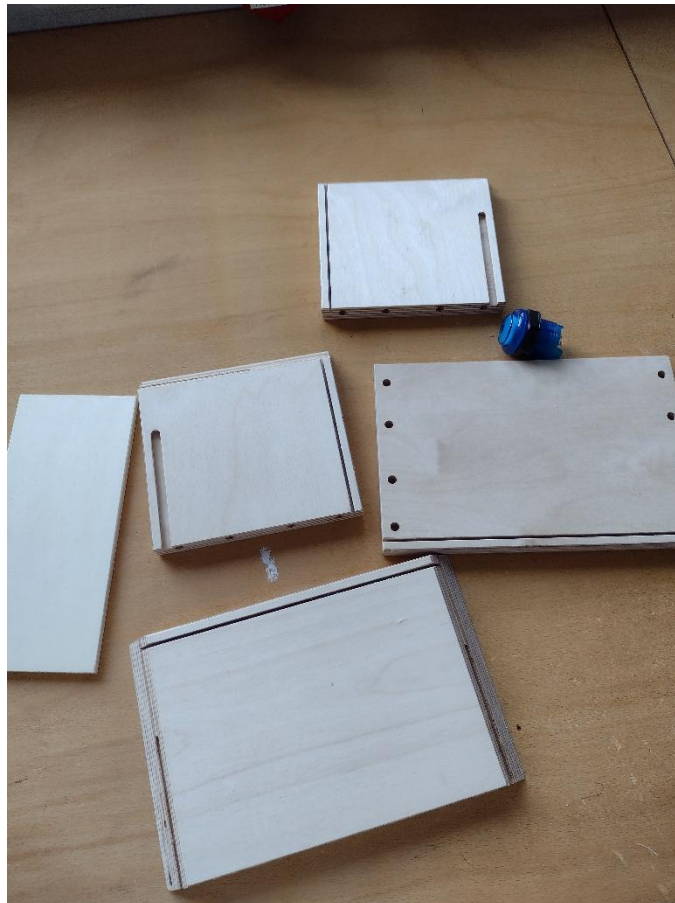
1x platform for the container

2x walls of the container (left and right)

1x back cover

1x roof

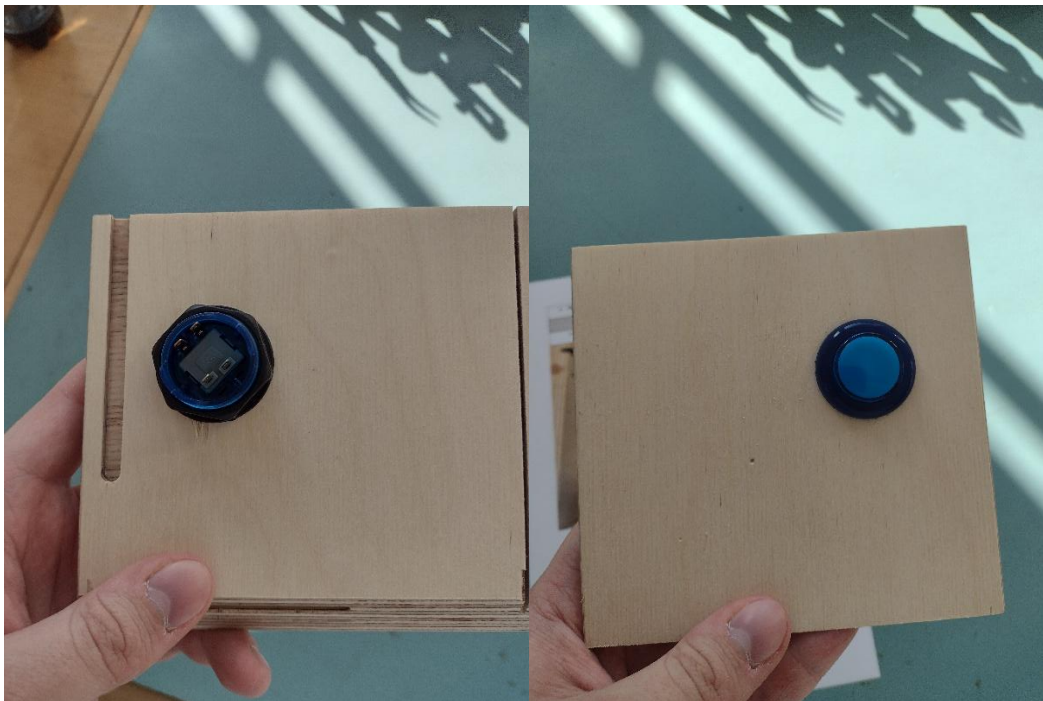
1x button



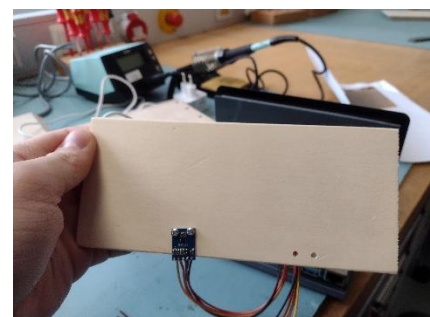
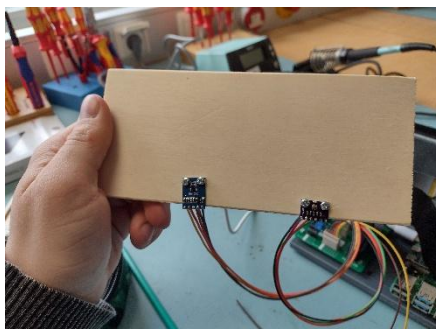
2. Holes:

Before assembling the container, we had to make some modifications to its walls. In the project documentation it was said that the walls have to have 1 hole for the button that is ~30mm in diameter and to have 1 hole for the LED wiring that is ~2.5mm in diameter. As the drilling stations were already prepared, the only thing that we had to do was to begin drilling.

Result(does not capture the LED wiring hole yet):



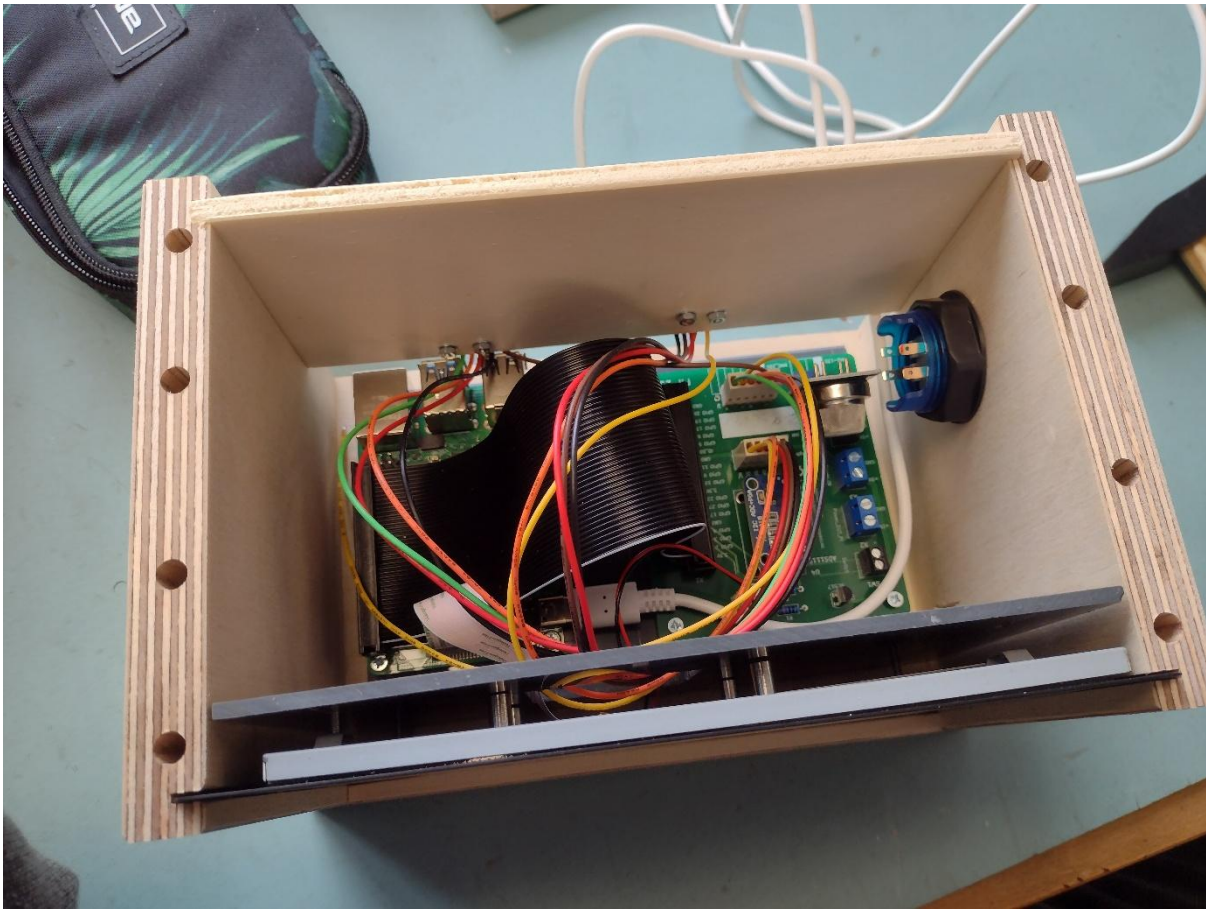
We also had to drill holes on the back plate, to allow us to let sensors hang out of the box to take measurements of the environment.



3. Assembly:

The container's parts already had predetermined holes drilled into it that allowed for a swift assembly; to connect platform and walls, we had to use hammer and dowels to connect the platform and walls. Then in the premade opening we slotted the backplate.

4. Result:



Userstory L5 – Data transfer

Table of Contents:

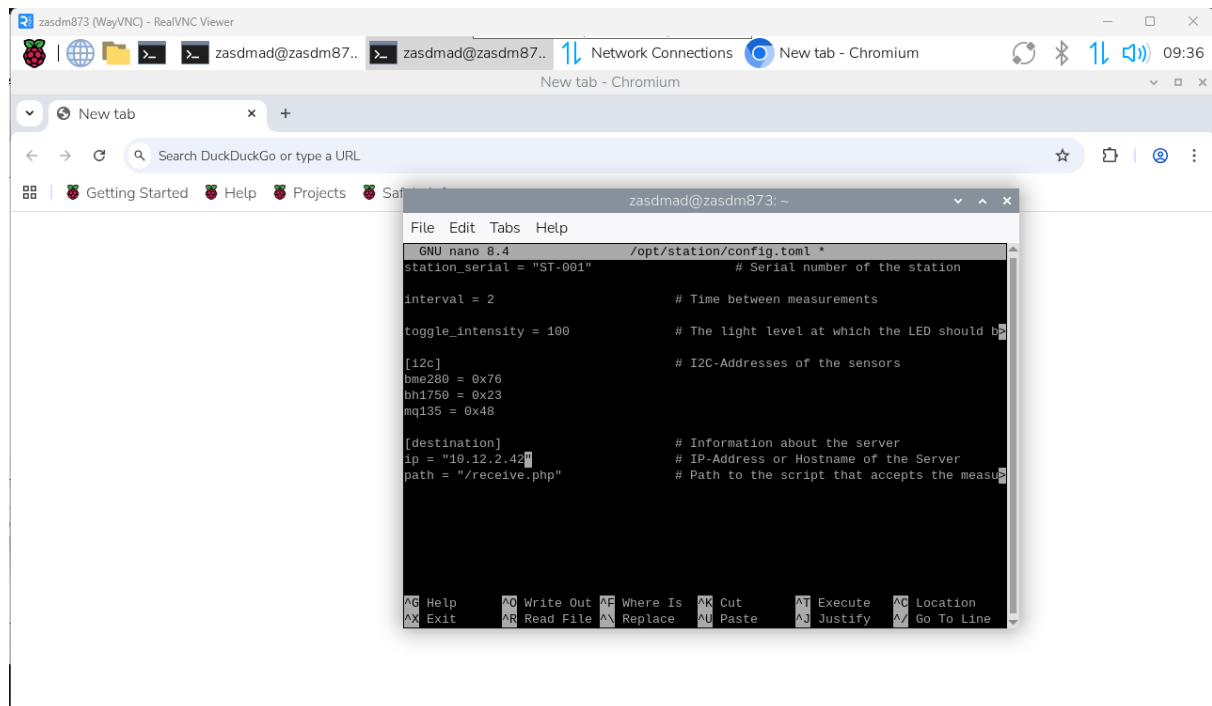
1. Introduction
2. Task achievement
3. Results

1. Introduction:

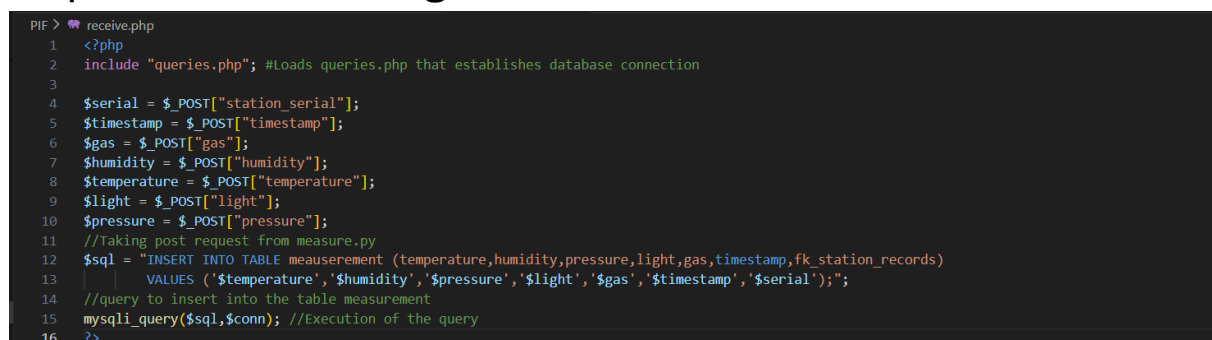
In this documentation will be included all the necessary steps to enable data transfer from Raspberry Pi to the webserver.

2. Task achievement:

To complete the task, first thing I had to do was to open config.toml file and change the ip with destination to the appropriate one in the server.



After that I had to create and upload receive.php on my webserver, that would process by taking it from the POST request and inserting it into the database.



Receive.php gets post requests sent by measure.py, then takes them as variables from superglobal array `$_POST[“”]` and then runs a sql script that is responsible for inserting data into the database.

Because my webserver is set on the virtual machine, I had to change the network settings and put it on the bridged connection, otherwise raspberry pi could not communicate with the webserver.

When I tried to run `measure.py` to test if the sending data works, it has shown me that python post request is being refused because the SSL certificate is self-signed. To fix it, I took down the https version of the website and when attempting to connect to the http version it did work.

3. Results:

In the end, raspberry pi was able to upload measurements to the web database server.

Server: localhost » Database: portableindoorfeedback » Table: measurement										
Browse Structure SQL Search Insert Export Import Privileges Operations More										
pk_measurement temperature humidity pressure light gas timestamp fk_station_record										
☐	Edit Copy Delete	4	18.90	60.00	1010.90	80.00	140.00	2025-12-01 09:55:00	ST-003	
☐	Edit Copy Delete	5	24.02	0.00	600.59	10.00	2560.00	2026-03-20 09:54:45	ST-001	
☐	Edit Copy Delete	6	22.21	0.00	985.58	10.83	2573.00	2026-03-20 09:54:47	ST-001	
☐	Edit Copy Delete	7	22.21	0.00	985.60	10.83	2574.00	2026-03-20 09:54:49	ST-001	
☐	Edit Copy Delete	8	22.20	0.00	985.61	10.83	2572.00	2026-03-20 09:54:51	ST-001	
☐	Edit Copy Delete	9	22.20	0.00	985.61	10.83	2587.00	2026-03-20 09:54:53	ST-001	
☐	Edit Copy Delete	10	22.20	0.00	985.62	10.83	2595.00	2026-03-20 09:54:55	ST-001	
☐	Edit Copy Delete	11	22.20	0.00	985.62	10.83	2620.00	2026-03-20 09:54:58	ST-001	
☐	Edit Copy Delete	12	22.20	0.00	985.63	10.83	2611.00	2026-03-20 09:55:00	ST-001	
☐	Edit Copy Delete	13	22.20	0.00	985.63	10.83	2589.00	2026-03-20 09:55:02	ST-001	
☐	Edit Copy Delete	14	22.20	0.00	985.63	10.83	2586.00	2026-03-20 09:55:04	ST-001	
☐	Edit Copy Delete	15	22.20	0.00	985.63	10.83	2589.00	2026-03-20 09:55:06	ST-001	
☐	Edit Copy Delete	16	22.20	0.00	985.62	10.83	2580.00	2026-03-20 09:55:08	ST-001	
☐	Edit Copy Delete	17	22.19	0.00	985.62	10.83	2573.00	2026-03-20 09:55:10	ST-001	
☐	Edit Copy Delete	18	22.19	0.00	985.62	10.83	2574.00	2026-03-20 09:55:12	ST-001	
☐	Edit Copy Delete	19	22.19	0.00	985.63	10.83	2579.00	2026-03-20 09:55:14	ST-001	
☐	Edit Copy Delete	20	22.19	0.00	985.63	10.83	2581.00	2026-03-20 09:55:16	ST-001	

PIF – Userstory A7

Table of Contents:

1. Introduction
2. Attaching the button
3. Attaching the LED strip
4. Result

1. Introduction:

In this userstory, I have attached a button that should power the LED lights and the LED lights.

List of equipment:

2x black wires

2x red wires

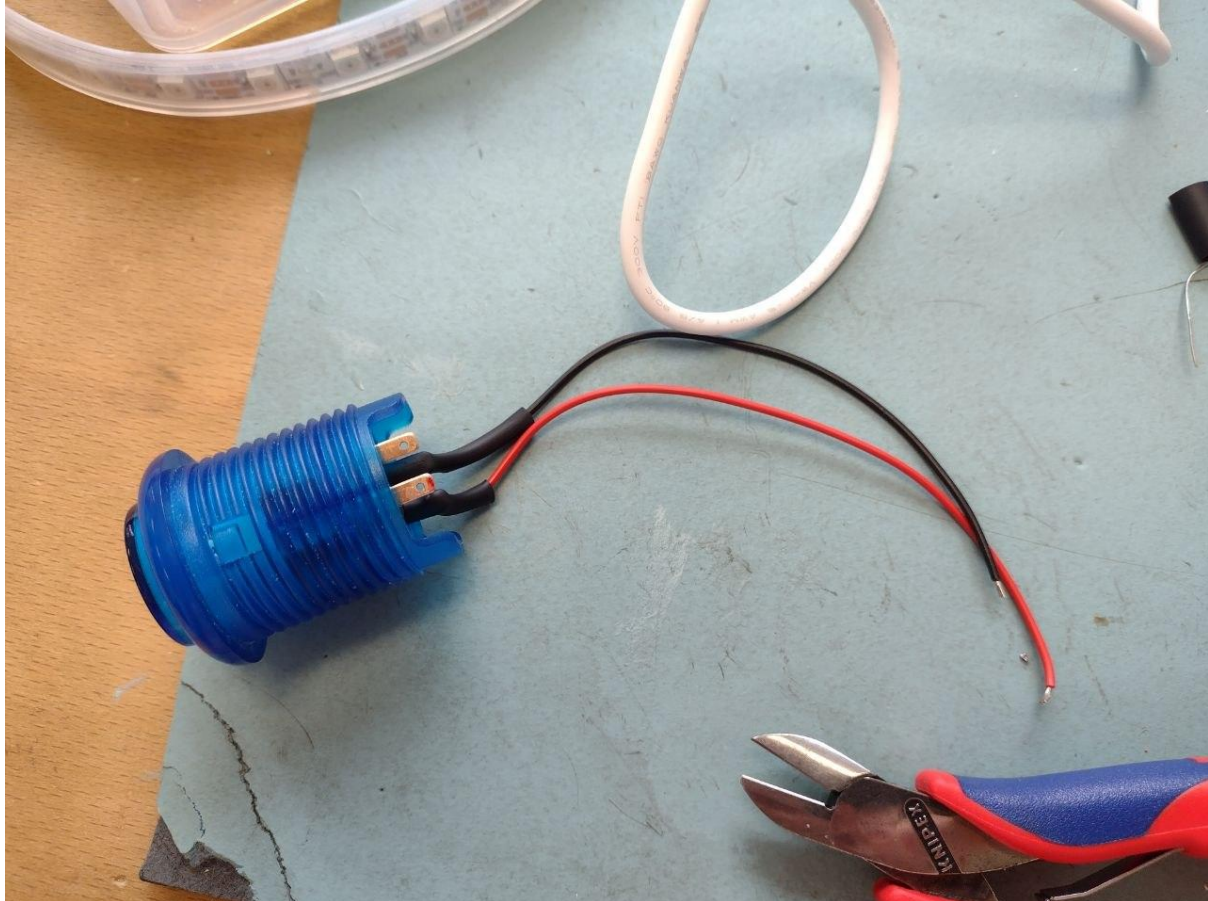
1x Soldering station

1x Roll of two-sided tape

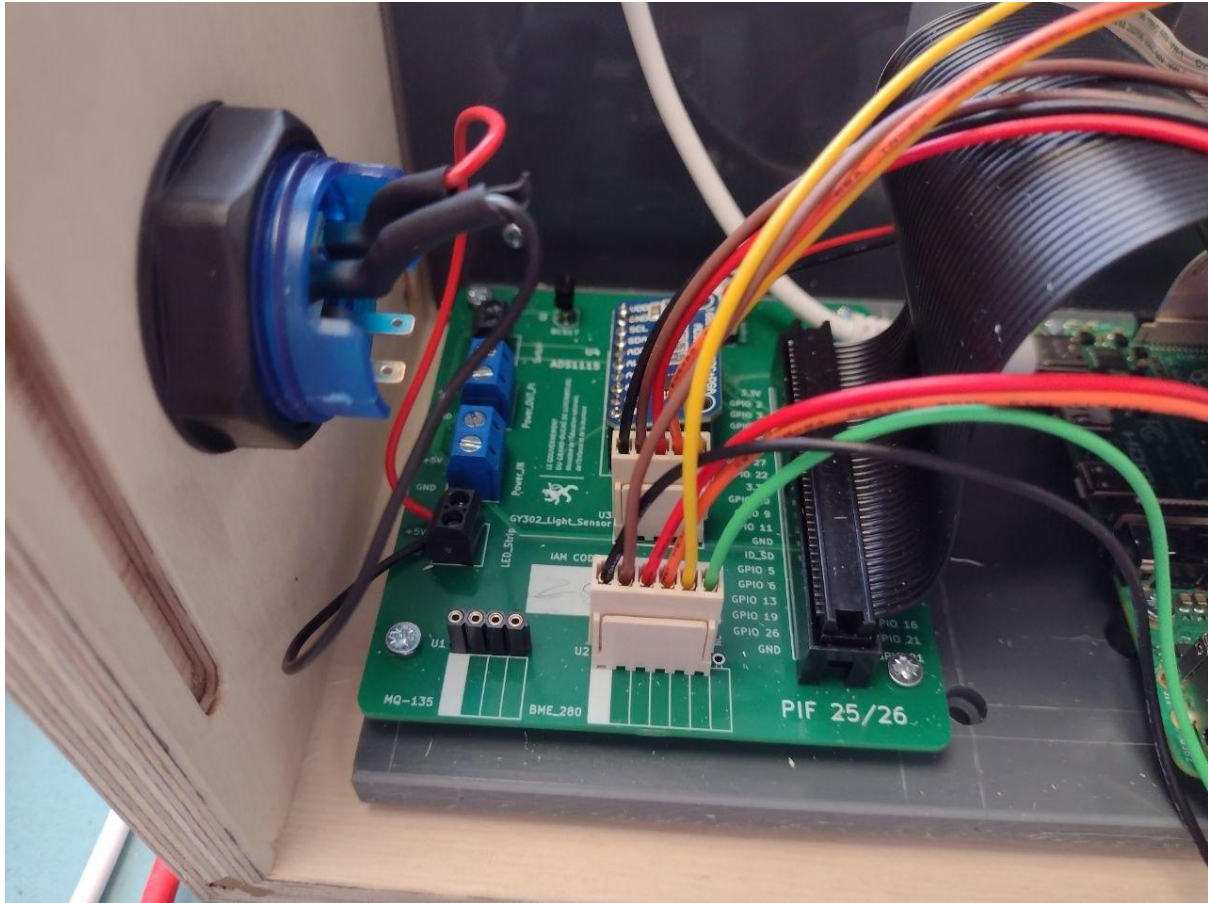
2x Shrinking tube

2. Attaching the button:

First I have soldered the black and red wires to the button, then I covered them with shrinking tube.



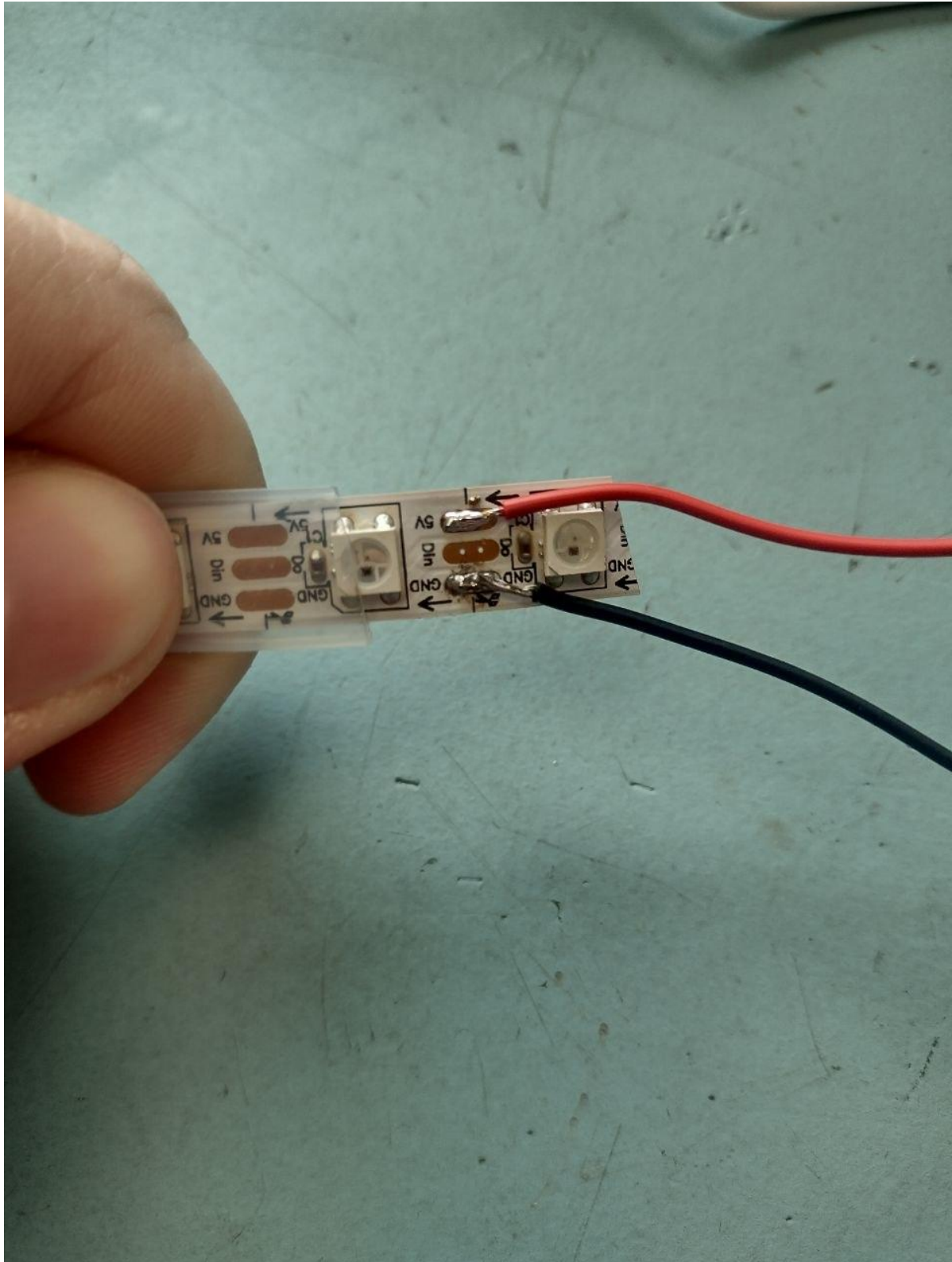
Then I slotted it in the printed circuit board.



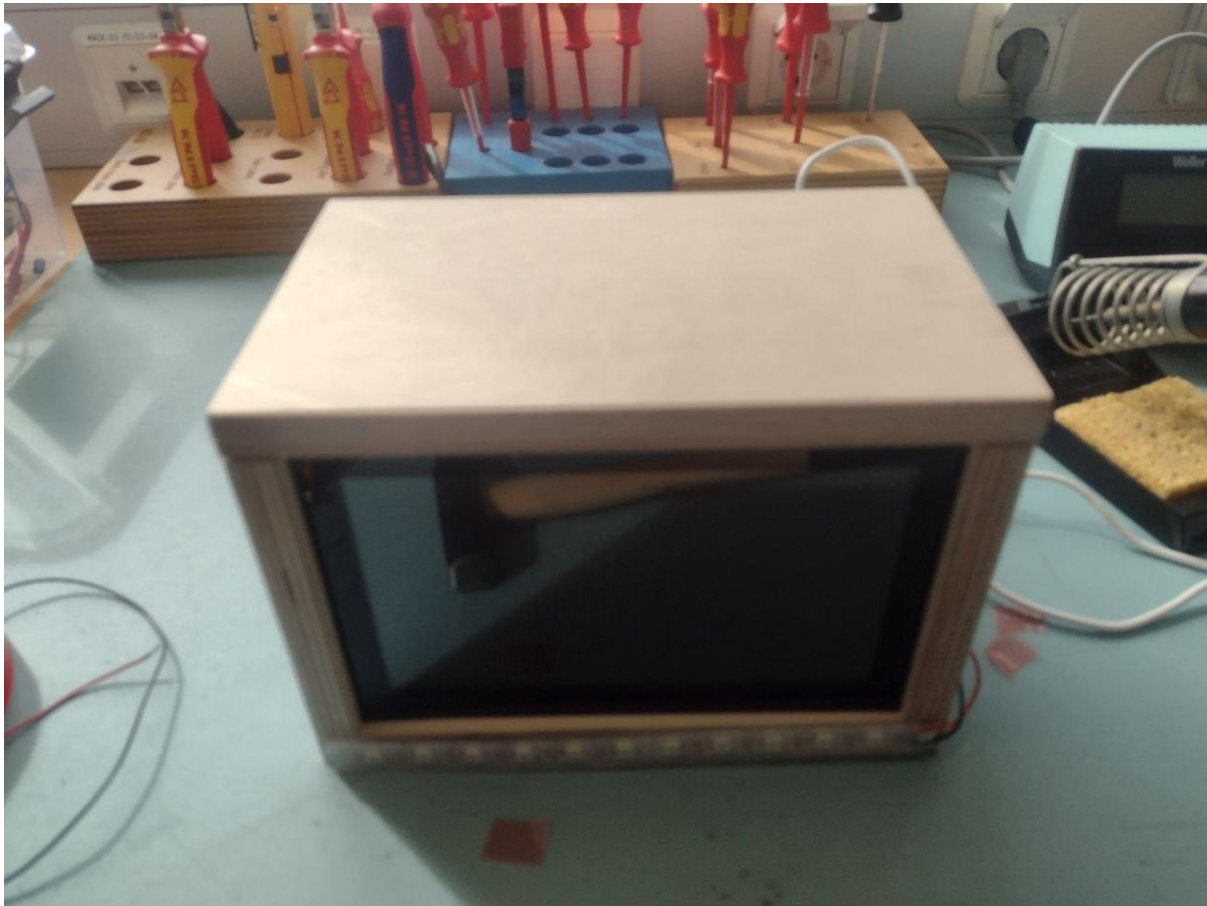
I have reattached the wires to the “switch” output later.

3. Attaching the LED:

Then, to attach the LED, I took another pair of black and red wires and soldered them to the LED strip.

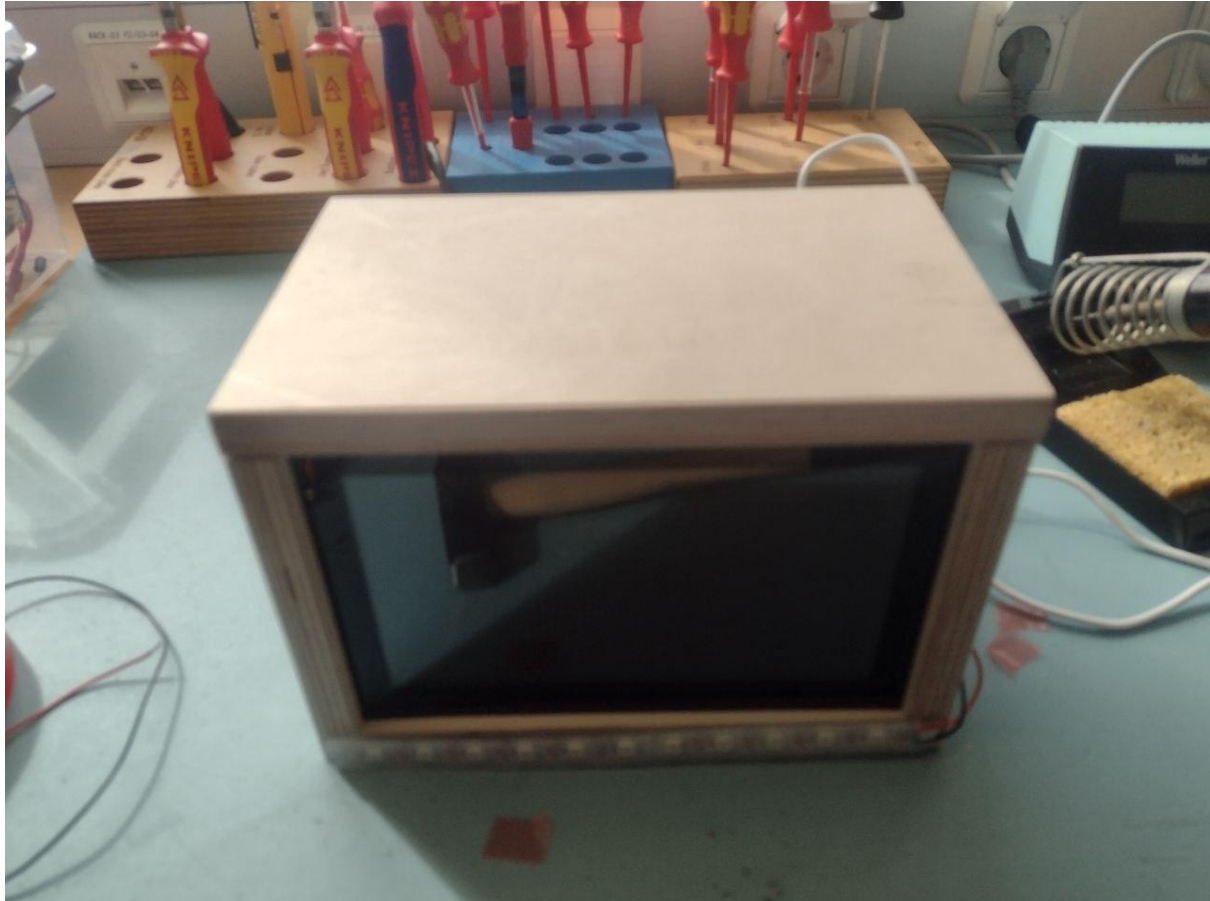


After that I have attached it to the “switch” output and taped the strip onto the box



4. Results:

I have managed to successfully attach the LED strip and Button to the printed circuit board and finished the assembly of the raspberry pi station.



Userstory L6 – Backup Server

Table of Contents:

- 1) Introduction
- 2) Securing MYSQL credentials
- 3) Creating backup script
- 4) Scheduling crontab job
- 5) Setting up FTP access
- 6) Data restoration process

1.Introduction

This documentation will provide necessary steps to ensure successful setup of the Backup Server running with Ubuntu Server OS. The first step in the beginning will be – installation of mysql-client and vsftpd. After this, the backupuser with home directory /backupuser needs to be created and structure created. Vsftpd will use /backupuser directory to provide backups. Cron is installed by default.

2. Securing MYSQL credentials

To secure MYSQL credentials, I used command `mysql_config_editor` with settings below:

```
zasdm873@pif-zasdm873-backup-2026:/$ mysql_config_editor set --login-path backup --host=portable-indoor-feedback.local --user=backupuser --password
Enter password:
```

After processing the command, it requests passwords and encrypts it, then it stores it in the `~/.mylogin.cnf` file that will keep those credentials.

Testing it with `mysql` command to execute database:

```
backupuser@pif-zasdm873-backup-2026:~$ mysql --login-path=backup -e "SHOW DATABASES;"
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| portableindoorfeedback |
| sys |
+-----+
```

3. Creating backup script:

To backup mysql data I have used mysqldump command.
To backup system data I have used sshpass to enter the system of my webserver and extract /var/www/ and /etc folders. At the bottom of the script is set automatic deletion of backups older than 7 days:

```
GNU nano 7.2 /usr/local/bin/backup.sh
#!/bin/bash
DATE=$(date +%F_%H-%M)
MYSQL_BACKUP_DIR="/backupuser/files/mysql"
SYSTEM_BACKUP_DIR="/backupuser/files/system"

mysqldump --login-path=backup portableindoorfeedback \
  | gzip > "$MYSQL_BACKUP_DIR/mysql_$DATE.sql.gz"

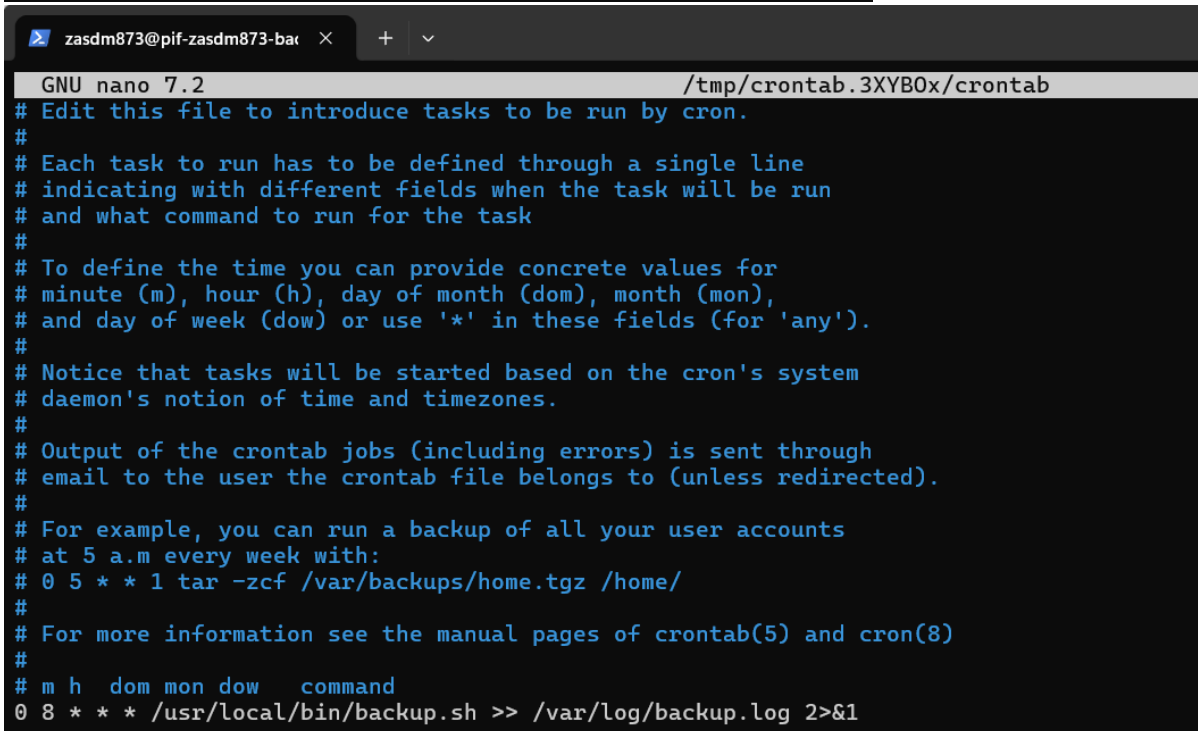
sshpass -p 'PIF' ssh backdoorbackup@'portable-indoor-feedback.local' \
  "tar -czf - /etc /var/www" \
  > "$SYSTEM_BACKUP_DIR/system_$DATE.tar.gz"
#tar -czf "$SYSTEM_BACKUP_DIR/system_$DATE.tar.gz" /etc /var/www

find /backupuser/files/* -type f -mtime +7 -delete
```

4. Scheduling crontab job:

To schedule a crontab job I used crontab environment nano.

```
zasdm873@pif-zasdm873-backup-2026:/$ crontab -e
no crontab for zasdm873 - using an empty one
crontab: installing new crontab
```



```
GNU nano 7.2 /tmp/crontab.3XYB0x/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 8 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

Then wrote this configuration. It activates every 8 hours /usr/bin/backup and directs it's output to /var/log/backup.log ignoring error output.

5. Setting up ftp access:

After installing vsftpd, I opened the config using nano

```
zasdm873@pif-zasdm873-backup-2026:~$ sudo nano /etc/vsftpd.conf
```

In the config file I have disabled anonymous ftp, allowed local enable, allowed write enable, allowed chroot_local_user

```
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
#
# Uncomment this to allow local users to log in.
local_enable=YES
# Uncomment this to indicate that vsftpd use a utf8 filesystem.
#utf8_filesystem=YES
write_enable=YES
chroot_local_user=YES
```

6. Data restoration process:

To retrieve the data stored in the backup server, I need to use ftp command and connect to the backup ip address.

```
zasdm873@zasdm873-PIF:~$ ftp backup
Connected to backup.
```

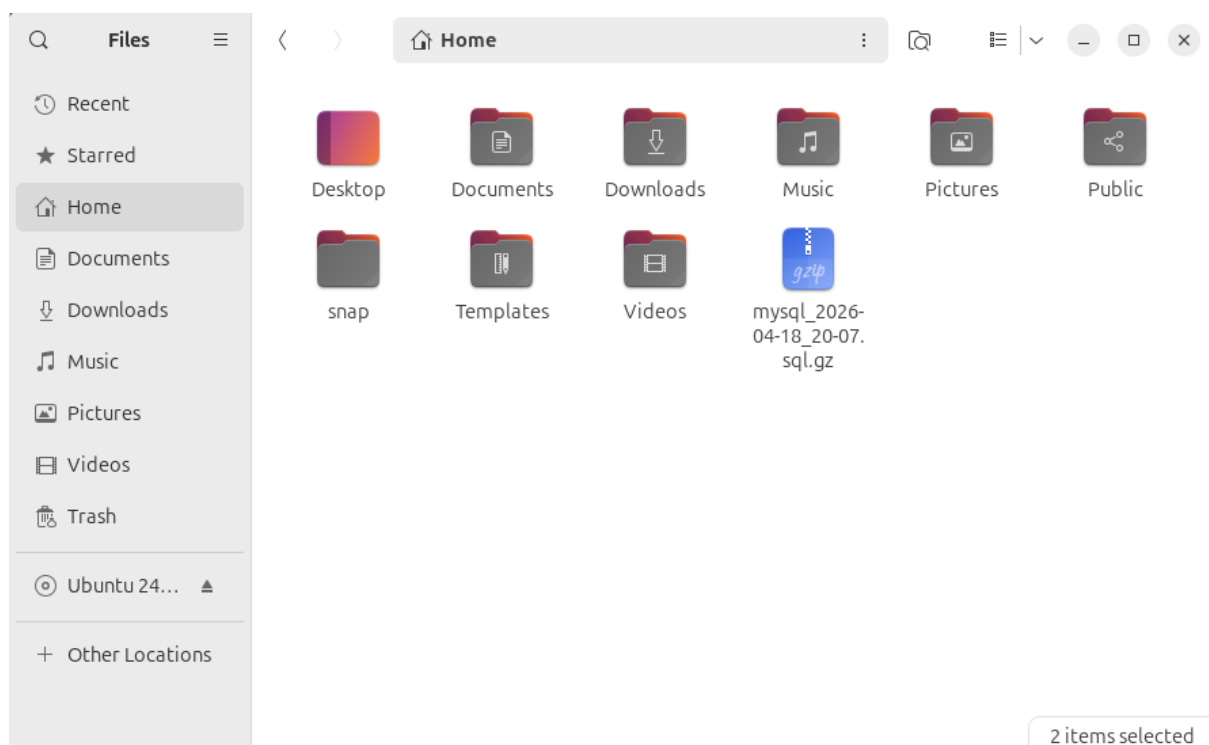
After that I need to login as a backup user.

```
Name (backup:zasdm873): backupuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd /files/mysql
```

After that, I need to go to the directory where data is stored use get <name of file> command to retrieve the backup.

```
ftp> cd files/mysql
250 Directory successfully changed.
ftp> get mysql_2026-04-18_20-07.sql.gz
local: mysql_2026-04-18_20-07.sql.gz remote: mysql_2026-04-18_20-07.sql.gz
229 Entering Extended Passive Mode (|||28569|)
150 Opening BINARY mode data connection for mysql_2026-04-18_20-07.sql.gz (35705 bytes).
100% |*****| 35705 3.89 MiB/s 00:00 ETA
226 Transfer complete.
35705 bytes received in 00:00 (3.37 MiB/s)
ftp>
```

Transfer successful



Now I will remove one of the entries in the database.

The first screenshot shows the phpMyAdmin interface for the 'portableindoorfeedback' database. The 'station' table is selected, and the query 'SELECT * FROM `station`' is executed. The table contains 7 rows. The row with 'pk_serialNumber' ST-007 (Garage Sensor) is selected for deletion.

pk_serialNumber	name	description	fk_user_owns
ST-001	Living Room Sensor	Measures air quality in living room	alice
ST-002	Bedroom Sensor	Indoor climate bedroom	bob
ST-003	Public Hall Sensor	Shared hallway sensor	NULL
ST-004	Test Station	Unassigned test station	NULL
ST-005	Kitchen Sensor	Kitchen air quality sensor	alice
ST-006	Office Sensor	Home office climate monitoring	alice
ST-007	Garage Sensor	Garage environmental sensor	bob

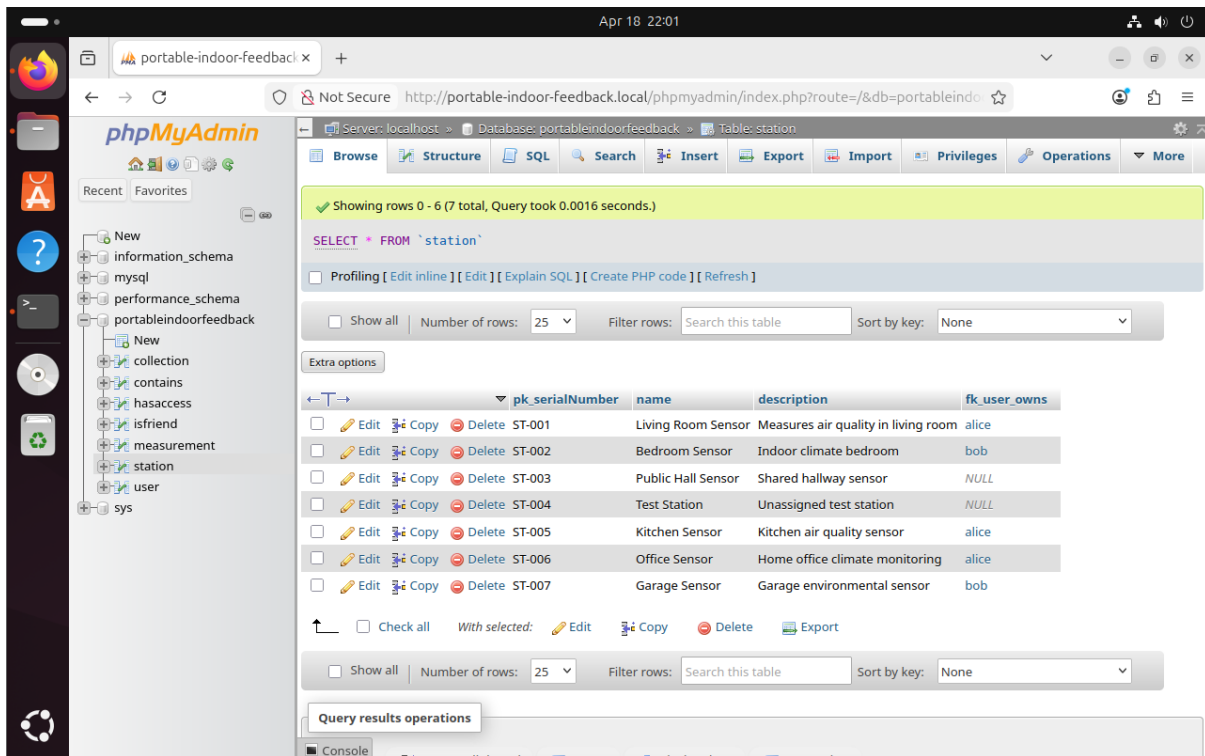
The second screenshot shows the same interface after the deletion of ST-007. The query results now show 6 rows, and ST-007 is no longer present in the table.

pk_serialNumber	name	description	fk_user_owns
ST-001	Living Room Sensor	Measures air quality in living room	alice
ST-002	Bedroom Sensor	Indoor climate bedroom	bob
ST-003	Public Hall Sensor	Shared hallway sensor	NULL
ST-004	Test Station	Unassigned test station	NULL
ST-005	Kitchen Sensor	Kitchen air quality sensor	alice
ST-006	Office Sensor	Home office climate monitoring	alice

To execute the backup I am using gunzip and pipe("|") to login into the mysql and apply the sqldump data to the database.

```
zasdm873@zasdm873-PIF:~$ gunzip < mysql_2026-04-18_20-07.sql.gz | mysql --login-path=backup portableindoorfeedback
```

After executing, refresh the page with the database and the backup is successful.



The screenshot shows the phpMyAdmin web interface. The browser address bar indicates the URL: `http://portable-indoor-feedback.local/phpmyadmin/index.php?route=/&db=portableindoorfeedback`. The interface shows the 'station' table selected in the 'portableindoorfeedback' database. The table structure is as follows:

pk_serialNumber	name	description	fk_user_owns
ST-001	Living Room Sensor	Measures air quality in living room	alice
ST-002	Bedroom Sensor	Indoor climate bedroom	bob
ST-003	Public Hall Sensor	Shared hallway sensor	NULL
ST-004	Test Station	Unassigned test station	NULL
ST-005	Kitchen Sensor	Kitchen air quality sensor	alice
ST-006	Office Sensor	Home office climate monitoring	alice
ST-007	Garage Sensor	Garage environmental sensor	bob